

iOS Fundamentals & Deep Understanding Roadmap

A bottom-up plan for moving from “vibe coding” to first-principles iOS engineering

Second edition, revised · 7 August 2026

10 paths · 52 chapters · 352 lessons · 52 chapter projects · 10 path capstones · 1 final project

Starting point	~1 year of professional iOS development experience
Core problem	APIs are familiar, but terminology and underlying mechanisms are fragmented
Goal	Connect machine → OS → compiler/runtime → Swift → UIKit → concurrency → networking/database → architecture
Method	Read fundamentals thoroughly, then trace each familiar iOS API down the stack
Commitment	15–20 hours per week for a full year, continuing into year two

North-star question: “What is actually happening underneath this line of code?”

What changed in this edition

The first edition set the structure and it holds. This edition changes five things, all of them decided by the learner after reading the first.

1. **The daily unit grew from twenty minutes to an hour.** The commitment went up to 15–20 hours a week, so the lesson grew to match.
2. **Four subjects were added** that the first edition left out: the design and composition gap, Interface Builder with runtime localization, gRPC over HTTP/2, and the device's sensors. Five new chapters, 38 new lessons.
3. **A language rule was added.** The learner reads English as a second language. The rule below says exactly how the writing must sound, and it bans the thing that was making explanations worse rather than better.
4. **A verification tier was added to every lesson.** No lesson may be built only from citation. Each one carries at least one number that was actually measured.
5. **Multiple choice was removed from the lesson contract.** Every question is now written and self-graded against a model answer. The lesson contract below explains why.

How to use this roadmap

This is not a beginner checklist and not a collection of disconnected courses. Treat it as a dependency graph. Learn enough of each lower layer to explain the upper layer you already use at work.

1. Start bottom-up with computer systems and operating systems.
2. Revisit Swift concepts and connect them to memory, compiler, runtime and ABI behaviour.
3. Revisit UIKit and concurrency with the run loop, scheduler, threads, rendering and isolation in mind.
4. Add networking, persistence, databases, testing, architecture and performance.
5. For every topic, write a one-paragraph “under the hood” explanation in your own words.
6. Prefer small experiments over passive reading when a concept feels abstract.

The rule. Do not ask only “How do I use this API?” Also ask: Who owns the memory? Which thread executes it? What OS service is involved? What compiler or runtime rule makes it work? What happens when it fails?

The whole stack at a glance

Layer	What you should understand	iOS connection
Hardware / machine	CPU, instructions, registers, memory, cache, storage	Why CPU work, memory pressure and locality affect app behaviour
Operating system	processes, threads, scheduling, virtual memory, files, syscalls	Why apps have lifecycles, sandboxes, main threads, background limits
Compiler / runtime	parsing, type checking, code generation, linking, runtime metadata	What Swift code becomes, and why ARC, generics and dispatch behave as they do
Swift	types, ownership, protocols, generics, errors, ARC	The language layer you write every day
UIKit / graphics	run loop, responder chain, layout, rendering, Core Animation	How taps become events and views become pixels
Design / composition	units, scales, colour, components, states, decomposition	What to ask the layout system for in the first place
Concurrency	tasks, threads, actors, queues, cancellation	How work is scheduled safely without freezing the UI
Networking	DNS, TCP, TLS, HTTP, HTTP/2, URLSession, gRPC	What actually happens during an API call
Persistence / DB	files, SQL, indexes, transactions, caches	Why different storage tools have different tradeoffs
Software engineering	architecture, testing, modularity, observability	How to make large apps understandable and maintainable

NEW IN THIS EDITION

Design and composition is now a layer of the stack. The first edition taught the constraint solver thoroughly and never taught what to ask it for. That is a real gap: you can know exactly how Auto Layout resolves a constraint and still be unable to look at a screen and see its structure. Path 4 now carries two chapters for it.

How the writing sounds

This roadmap and every lesson written from it follow one language rule. It exists because the learner reads English as a second language, and because the first attempt at “making it easier” made it worse.

- **Keep every technical term.** `retain cycle`, `mmap`, `witness table`, `page fault` stay exactly as they are. Renaming a term teaches the wrong word and makes the real documentation unreadable later. The terminology is the syllabus, not an obstacle to it.
- **Simplify the ordinary words around the terms.** “the thing that decides”, not “the arbiter”. “runs later”, not “is deferred until such time as”. The hard part was never `mmap`. It was the sentence `mmap` was buried in.
- **Short sentences.** Twenty-five words on average, thirty-five at the absolute most, one idea per sentence. No semicolon chains.
- **No folksy analogies.** Explaining memory by comparing it to a car dealership or a post office teaches the dealership. Then the comparison has to be unlearned, because it breaks at exactly the point where the real behaviour gets interesting.
- **Examples are building examples.** Every example is something being constructed: code you would actually write, a screen you would actually ship, a measurement you would actually take.
- **Draw the real machine, not a comparison to one.** When a picture is needed, it is a memory diagram, a byte table, or a captured frame dump.

Verification tiers — the gate against trivia

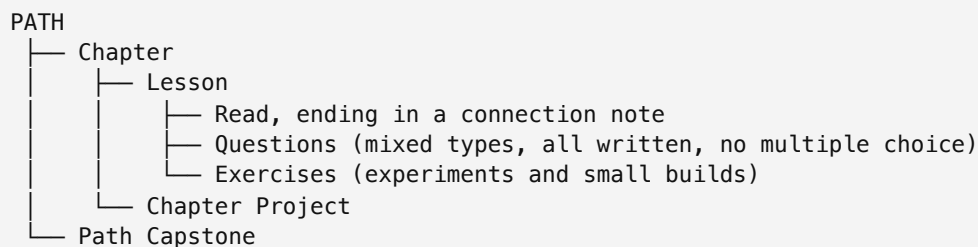
Every lesson is tagged with how its claims were checked.

Tier	Meaning
EXEC	Runnable and run. Plain <code>swift</code> , <code>Mac Catalyst</code> , <code>ibtool</code> , command-line tools. The numbers in the lesson were measured, not remembered.
DEV	Needs a real device — permission prompts, the render server, biometrics.
DOC	Cited convention or documented behaviour that cannot be measured directly.

A lesson that is entirely DOC is a failed lesson. Every lesson must carry at least one measured number. This is the defence against the whole thing decaying into recitable trivia — “minimum touch target? 44 points” is a fact you can hold without understanding anything. The measurement around it is not.

Curriculum architecture

Every path has chapters. Every chapter has lessons. Every lesson is learned actively. Every chapter ends with a build. Every path ends with a capstone. The whole program ends with one production-style project.



ALL PATHS → Final Integrated iOS Project

The lesson contract — what fills the hour

REVISED

Multiple choice is gone. A four-option question can be passed by recognizing the right shape among the choices, without ever pulling the idea out of memory — exactly the “looks familiar” failure below warns against. Every lesson now has two practice parts. Both are written. Both are graded by rating your own answer against a model answer, the same honesty contract explain-back always used. There is nothing left to guess.

Part	Minimum output	Time
Read	Understand the terminology, the mechanism, the purpose, the tradeoffs and the common failure modes. Ends with a connection note tracing the lesson up to iOS and down to the machine.	~15 min
Questions	5–8 open-response questions, mixed types: recall (define a term), reasoning (argue a tradeoff), trace (predict output or state before running anything), debug (find and fix a broken scenario), explain (the whole concept, no notes), apply (use it on a fresh scenario).	~25 min
Exercises	1–3 focused experiments or small builds, each small enough to isolate one concept. Written without AI. Gate the pass — a lesson cannot be marked done with one left undone.	~20 min

Do not grade yourself by recognition. A lesson is not complete because the explanation looks familiar. It is complete when you can retrieve the terminology, predict behaviour, build a small example, debug a failure, and connect it to adjacent layers without relying on AI.

Chapter project contract

- Combines several lessons from the chapter rather than testing one API in isolation.
- Contains at least one deliberate failure or debugging task.
- Requires a short design note explaining decisions and tradeoffs.

- Uses real tooling where relevant: debugger, Memory Graph, Instruments, network logs, database inspection, tests.
- Ends with a “trace it downward” explanation connecting the build to lower layers.

Path capstone contract

- Combines all major chapters in that path.
- Buildable without following a step-by-step tutorial.
- Includes tests or verification where appropriate.
- Includes at least one performance, memory, concurrency or failure analysis.
- Produces a concise write-up: architecture, mechanisms, tradeoffs, bugs found, and what happens underneath.

Projects and capstones are written with zero AI assistance. That rule is part of the method, not a preference. Reading can be assisted; producing cannot, because the whole point is to find out what you can build without help.

Path o — Computer Systems (28 lessons)

Build the mental model every later topic rests on. Not electrical-engineering depth — enough to explain where instructions and data live, and how a program becomes running work.

Every lesson here answers: What data is represented in memory? Which operations happen in the CPU, registers and cache? What changes if the data layout or access pattern changes? Can I predict the program state before running it?

oA — Data representation

01	Bits and bytes	EXEC
02	Binary and hexadecimal	EXEC
03	Signed integers and two's complement	EXEC
04	Floating point, and the precision surprise	EXEC
05	Text: ASCII, Unicode, UTF-8	EXEC
06	Endianness	EXEC

Project — Binary inspector. Encode and decode values by hand and in code, inspect the bytes, and demonstrate an overflow and a floating-point precision surprise.

oB — CPU and instructions

01	What a CPU actually does	DOC
02	Registers and the program counter	EXEC
03	The instruction cycle: fetch, decode, execute	DOC
04	Reading the assembly your Swift becomes	EXEC
05	Cores, and why more of them means parallel work	EXEC
06	Cache and locality	EXEC
07	Context switching, and what it costs	EXEC

Project — Execution microscope. Compile tiny functions, inspect the stack and disassembly in the debugger, and time a cache-friendly loop against a cache-hostile one.

oC — Memory

01	Addresses and pointers	EXEC
02	RAM versus persistent storage	EXEC
03	The stack	EXEC
04	The heap	EXEC
05	Allocation and deallocation	EXEC
06	Copying a value versus sharing a reference	EXEC
07	Fragmentation	EXEC
08	The memory hierarchy, in real numbers	EXEC

Project — Memory laboratory. Stack traces, heap allocations, a large buffer, object lifetime, and a memory-growth experiment you can explain line by line.

oD — Program execution

01	What an executable file is	EXEC
02	Code, data, and the other sections	EXEC
03	What happens when a program is loaded	EXEC
04	The function call	EXEC
05	The stack frame	EXEC
06	The return address	EXEC
07	Crashes: bad access, stack overflow, out of memory	EXEC

Project — Program trace. Document one small program from source to executable to process to function call to memory to crash.

Path capstone — Systems Trace Lab. Take a small program and produce a source-to-CPU/memory execution map with debugger observations.

Path 1 — Operating Systems (28 lessons)

The environment that owns processes, threads, virtual memory, files, sockets, permissions, scheduling and application lifecycle.

Every lesson here answers: Which process or thread is involved? Is the work running, blocked, sleeping or waiting on I/O? What resource or permission does the OS own? What happens under memory pressure or a lifecycle change?

1A — Processes and threads

01	Program versus process	EXEC
02	The process address space	EXEC
03	What a thread is	EXEC
04	The main thread	EXEC
05	User mode versus kernel mode	DOC
06	System calls	EXEC
07	How a process starts, and how it ends	EXEC

Project — Process explorer. Inspect process and thread state, and trace one blocking operation all the way down.

1B — Scheduling

01	The scheduler	DOC
02	Runnable, running, waiting	EXEC
03	Priority and quality of service	EXEC
04	CPU-bound versus I/O-bound work	EXEC
05	Why blocking the main thread freezes the screen	EXEC
06	What a context switch costs	EXEC

Project — Responsive-work demo. Compare blocking UI work against scheduled background work, and explain the scheduler's part in both.

1C — Virtual memory

01	Virtual versus physical memory: the address is a lie	EXEC
02	Pages, and the fault that fills them	EXEC
03	Memory protection	EXEC
04	Copy-on-write, at the page level	EXEC
05	mmap: why a 300 MB file is not 300 MB of RAM	EXEC
06	Clean versus dirty, and why iOS has no swap	EXEC
07	Memory pressure and jetsam	DOC
08	Measuring it: footprint, resident, virtual	EXEC

Project — Memory-pressure lab. Allocations, copy behaviour, warning and termination concepts, and a measurement you trust.

1D — Files and the iOS process environment

01	File descriptors	EXEC
02	Directories, paths, and the app container	EXEC
03	Buffered I/O and flushing	EXEC
04	File permissions and metadata	EXEC
05	Atomic replacement	EXEC
06	The app sandbox	DOC
07	App lifecycle, suspension, background limits	DEV

Project — Sandboxed storage app. Read and write files in the correct containers, and document the lifecycle and background constraints you hit.

Path capstone — OS Behavior Explorer. A small iOS diagnostic app demonstrating lifecycle, threads, files, memory pressure and background limits.

Path 2 — Compiler, Linker and Runtime (26 lessons)

What happens between Swift source and machine instructions, and where compile-time behaviour ends and runtime behaviour begins.

Every lesson here answers: What is decided at compile time? What is left for link, load or runtime? Which symbol, type or metadata is involved? What ownership or dispatch mechanism explains the behaviour?

2A — The compilation pipeline

01	What a compiler actually is	DOC
02	Lexing: source text into tokens	EXEC
03	Parsing, and the AST	EXEC
04	Type checking	EXEC
05	SIL and IR: the middle languages	EXEC
06	Optimization	EXEC
07	Machine-code generation	EXEC

Project — Compiler-errors notebook. Create syntax, type and generic errors on purpose, and classify which phase rejected each one.

2B — Linking and loading

01	Object files	EXEC
02	Symbols and name mangling	EXEC
03	Static libraries	EXEC
04	Dynamic libraries and frameworks	EXEC
05	What the linker does	EXEC
06	Linker errors versus compiler errors	EXEC
07	dyld, and what load time costs	EXEC

Project — Binary dependency map. Inspect a real dependency graph and trace one symbol to the code that defines it.

2C — Runtime mechanisms

01	Runtime metadata	EXEC
02	Static versus dynamic dispatch	EXEC
03	Vtables	EXEC
04	Protocol witness tables	EXEC
05	The Objective-C runtime	EXEC
06	ABI and module stability	DOC

Project — Dispatch lab. Compare `final`, `class`, `protocol` and `@objc` calls, and prove which mechanism each one used.

2D — ARC and ownership machinery

01	Reference counting	EXEC
02	Where <code>retain</code> and <code>release</code> are inserted	EXEC
03	<code>deinit</code> , and deterministic destruction	EXEC
04	Closure capture, and the context box	EXEC
05	<code>weak</code> and <code>unowned</code>	EXEC
06	Retain cycles, and how to prove one	EXEC

Project — Ownership debugger. Create a leak on purpose, prove it with `deinit` and the Memory Graph, then fix it.

Path capstone — Source-to-Runtime Report. Pick one Swift feature and trace it from syntax through compile, link, load and runtime behaviour.

Path 3 — Swift (39 lessons)

You already write Swift. This path removes blind spots, installs precise terminology, and connects syntax to semantics.

Every lesson here answers: What is the exact type? Value or reference semantics? Who owns this? What does the compiler guarantee, and what can still fail at runtime?

3A — Language core

01	let and var	EXEC
02	The primitive types, and their sizes	EXEC
03	Operators	EXEC
04	Control flow	EXEC
05	Optionals	EXEC
06	Optional is an enum	EXEC
07	Tuples and ranges	EXEC

Project — Swift fundamentals kata pack. Transformations, optional handling, collection reasoning and prediction exercises.

3B — Functions and closures

01	Functions and argument labels	EXEC
02	Default, inout and variadic parameters	EXEC
03	Functions as values	EXEC
04	What a closure physically is	EXEC
05	Captures	EXEC
06	escaping	EXEC
07	Capture lists	EXEC
08	map, filter, reduce	EXEC

Project — Callback engine. Design synchronous and escaping callback APIs, then diagnose the capture and lifetime bugs you built into them.

3C — Type modeling

01	Structs	EXEC
02	Classes	EXEC
03	Value versus reference semantics: the decision	EXEC
04	Enums	EXEC
05	Enums with payloads	EXEC
06	Stored, computed and observed properties	EXEC
07	Initializers and inheritance	EXEC
08	final, and identity	EXEC

Project — Domain model project. Model one real feature with value and reference types, and justify every choice in writing.

3D — Protocols and generics

01	Protocols	EXEC
02	Extensions and default implementations	EXEC
03	Protocol composition	EXEC
04	Associated types	EXEC
05	Generic functions	EXEC
06	Generic types and constraints	EXEC
07	where clauses	EXEC
08	any versus some: the existential box	EXEC

Project — Generic component library. Reusable repository, result and collection utilities behind protocol abstractions.

3E — Errors, ownership and standard protocols

01	Error and throws	EXEC
02	do, catch, and the second return path	EXEC
03	try? and Result	EXEC
04	Equatable	EXEC
05	Hashable	EXEC
06	Comparable	EXEC
07	Type casting and metatypes	EXEC
08	Copy-on-write in the standard library	EXEC

Project — Robust model layer. Typed errors, hashable identity, ownership-safe callbacks, and tests.

Path capstone — Swift Core Library. A tested Swift package or CLI using protocols, generics, errors, value semantics, reference ownership and documentation.

Path 4 — UIKit and Interface (61 lessons)

Master the public API surface, then the machinery that explains lifecycle, layout, events, rendering and performance. This edition adds the part the first edition missed: how to look at a screen and see its structure.

Every lesson here answers: Where is this object in the hierarchy? Which lifecycle, layout or event phase is active? Why must this happen on the main thread? What work reaches Core Animation and the GPU?

4A — Views and controllers

01	UIView	EXEC
02	The view hierarchy	EXEC
03	The view controller lifecycle	EXEC
04	View loading, and <code>isViewLoaded</code>	EXEC
05	The controls you use every day	EXEC
06	Storyboard versus XIB versus programmatic	EXEC

Project — Lifecycle laboratory. A multi-screen app logging creation, appearance, disappearance, deinit and every hierarchy change.

4B — Layout

01	frame, bounds, center	EXEC
02	Coordinate conversion	EXEC
03	Auto Layout: constraints are equations	EXEC
04	The layout pass: mark now, solve later	EXEC
05	Safe areas	EXEC
06	Intrinsic content size	EXEC
07	Hugging and compression resistance	EXEC
08	Self-sizing cells	EXEC

Project — Adaptive layout project. One complex screen supporting Dynamic Type and rotation with zero ambiguous constraints.

4C — Navigation and containers

01	UINavigationController	EXEC
02	Tab bars	EXEC
03	Presentation	EXEC
04	Child view controllers	EXEC
05	Appearance transitions	EXEC

Project — Navigation shell. A coordinator-ready multi-flow app whose navigation stack you can inspect and explain.

4D — Lists and state-driven UI

01	UITableView	EXEC
02	UICollectionView	EXEC
03	Cell reuse	EXEC
04	Delegates and data sources	EXEC
05	Diffable data sources	EXEC
06	Compositional layout	EXEC

Project — Large-list browser. A pagination-ready list with diff updates, stable identity and preserved scroll position.

4E — Events and the run loop

01	What an event loop is	EXEC
02	The main run loop, and its modes	EXEC
03	UIResponder and the responder chain	EXEC
04	First responder	EXEC
05	Touch delivery	EXEC
06	Hit-testing	EXEC
07	Gesture recognizers and target-action	EXEC

Project — Event inspector. Custom views that log and demonstrate hit-testing and responder-chain behaviour.

4F — Rendering, animation and accessibility

01	UIView versus CALayer	EXEC
02	The Core Animation layer tree	EXEC
03	Layout, display, compositing	DOC
04	CPU versus GPU responsibilities	DOC
05	Offscreen rendering	DEV
06	draw(_:) and Core Graphics	EXEC
07	Animation: two layers, one lie	EXEC
08	VoiceOver and Dynamic Type	DEV

Project — Accessible animated interface. Custom animation and drawing with correct VoiceOver semantics and a performance inspection.

NEW IN THIS EDITION

4G and 4H exist because of one sentence: “if I see a UI, the layout should come into my mind, but it doesn’t.” Every other chapter in this path teaches the machinery that executes a layout. These two teach what to ask it for. 4G is the material a design is made from; 4H is how to take a screen apart.

4G — Design fundamentals

01	The point is not the pixel	EXEC
02	The spacing scale	EXEC
03	The type scale, and Dynamic Type	EXEC
04	Colour, contrast, and the two themes	EXEC
05	Material, elevation, and what a shadow costs	EXEC
06	Icons and SF Symbols	EXEC
07	Images, and the decode tax	EXEC

Project — Design token sheet. Build one screen twice: once with ad-hoc numbers, once from a spacing and type scale you defined. Measure both at the largest Dynamic Type setting and show which one survives.

4H — Composition and components

01	Reading a screen as a tree	EXEC
02	Stacks are the decomposition tool	EXEC
03	The component, and its states	EXEC
04	Lists, rhythm, and the repeated row	EXEC
05	The five screen states: loading, loaded, empty, error, offline	EXEC
06	Touch targets and reachability	EXEC
07	When a design system earns its keep	EXEC

Project — Screen teardown and rebuild. Take a screenshot of a screen you did not design. Draw its view tree, name every component and state, then rebuild it from your own tokens without looking at the original while you code.

4I — Interface Builder and localization

01	What a storyboard actually is	EXEC
02	Segues versus manual instantiation	EXEC
03	XIBs, reusable views, and File's Owner	EXEC
04	Storyboards at scale, and the merge conflict	EXEC
05	Traits and size classes	EXEC
06	Localization under the hood: tables, plurals, direction	EXEC
07	Switching language at runtime	EXEC

Project — Two-language screen. One storyboard screen, fully localized, including a plural rule and a right-to-left language, switching language without an app restart.

Path capstone — UIKit Application. A polished multi-screen app with adaptive layout, lists, navigation, accessibility, custom interaction and Instruments checks.

Path 5 — Concurrency (34 lessons)

Every lesson here answers: Task or thread? Blocking or suspension? What mutable state is shared? What ordering is guaranteed? How do cancellation, errors and isolation propagate?

5A — Execution fundamentals

01	Concurrency versus parallelism	EXEC
02	Synchronous versus asynchronous	EXEC
03	Thread versus task	EXEC
04	Blocking versus suspension	EXEC
05	Race conditions and data races	EXEC
06	Deadlock, livelock, starvation	EXEC

Project — Race and freeze lab. Freeze the UI on purpose, create an ordering bug and a race, then explain each failure.

5B — GCD

01	DispatchQueue	EXEC
02	Serial versus concurrent queues	EXEC
03	The main and global queues	EXEC
04	async versus sync	EXEC
05	Quality of service	EXEC
06	Groups, semaphores and barriers	EXEC
07	Why main.sync deadlocks	EXEC

Project — Legacy concurrency project. An image or data pipeline built on queues, with a safe handoff to the UI.

5C — async/await and Task

01	async functions	EXEC
02	await is a suspension point	EXEC
03	async throws	EXEC
04	Task	EXEC
05	Task lifetime	EXEC
06	Cancellation is cooperative	EXEC
07	Priorities and sleep	EXEC

Project — Cancellable API screen. Loading, error, retry and cancel behaviour with navigation-safe task lifetime.

5D — Structured concurrency

01	Parent and child tasks	EXEC
02	async let	EXEC
03	Task groups	EXEC
04	How errors and cancellation propagate	EXEC

Project — Concurrent downloader. Fetch a dynamic set of resources with bounded reasoning about completion and failure.

5E — Actors and isolation

01	Actors	EXEC
02	Actor isolation	EXEC
03	Reentrancy	EXEC
04	MainActor	EXEC
05	nonisolated	EXEC
06	Sendable, and the Swift 6 checks	EXEC

Project — Thread-safe cache. Start with unsafe mutable state, demonstrate the bug, then fix it with actor isolation.

5F — Streams and bridging

01	AsyncSequence	EXEC
02	AsyncStream	EXEC
03	Continuations	EXEC
04	Migrating completion handlers	EXEC

Project — Event-stream adapter. Wrap a callback or notification stream as an AsyncSequence and consume it safely.

Mental model. A Swift Task is not an OS thread. Tasks are scheduled by the Swift concurrency runtime onto threads, which the OS then schedules onto CPU cores.

Path capstone — Concurrency Workbench. A UIKit app with parallel network work, cancellation, an actor-protected cache, streaming updates, error propagation and a thread analysis.

Path 6 — Networking (42 lessons)

Every lesson here answers: Which protocol layer is responsible? What bytes, headers and status actually crossed? What is retryable and what is not? Where should caching, auth and cancellation live?

6A — Network foundations

01	Packets	DOC
02	IP addresses	EXEC
03	Ports	EXEC
04	DNS	EXEC
05	TCP: the pipe under HTTP	EXEC
06	UDP, and when it wins	EXEC
07	Latency versus bandwidth	EXEC

Project — Network trace notebook. Follow one hostname from DNS lookup to an open socket, with captured evidence at each step.

6B — TLS and HTTP

01	What TLS is for	EXEC
02	Certificates and the chain	EXEC
03	The handshake, and what it costs	EXEC
04	Request and response	EXEC
05	Methods	EXEC
06	Status codes	EXEC
07	Headers and body	EXEC
08	Caching headers	EXEC
09	Cookies, tokens and idempotency	EXEC

Project — HTTP laboratory. Issue requests, inspect headers, status, body and cache behaviour, and classify every failure you can produce.

6C — URLSession and decoding

01	URL and URLRequest	EXEC
02	URLSession and its configurations	EXEC
03	The async URLSession APIs	EXEC
04	Codable	EXEC
05	Inside JSONDecoder	EXEC
06	Decoding resilience	EXEC
07	Timeouts and cancellation	EXEC

Project — Typed API client. An endpoint abstraction with decoding, a typed error model and working cancellation.

6D — Production networking

01	Pagination	EXEC
02	Retries and backoff	EXEC
03	Rate limits	EXEC
04	Auth refresh, and the single flight	EXEC
05	Request deduplication	EXEC
06	Where caching belongs	EXEC
07	Offline behaviour	EXEC
08	WebSockets and background transfer	EXEC

Project — Resilient feed. A paginated UI with a retry policy, deduplication, auth handling and cache integration.

NEW IN THIS EDITION

6E covers gRPC over HTTP/2, which the first edition left out entirely. It is in because it is used daily at work, and because REST is not the only thing on the wire. This chapter is where “what actually crossed the network” stops being an abstraction: you encode a message by hand and check it byte for byte.

6E — Protocols on the wire

01	Protocol Buffers: the wire format	EXEC
02	.proto evolution: changing the contract safely	EXEC
03	HTTP/2: frames, streams, multiplexing	EXEC
04	Anatomy of a gRPC call	EXEC
05	Deadlines and cancellation	EXEC
06	Streaming RPCs and flow control	EXEC
07	Interceptors	EXEC
08	SwiftNIO and transports	EXEC
09	TLS and certificate pinning	EXEC
10	Connection lifecycle: keepalive, backoff, waiters	EXEC
11	Auth over gRPC: tokens, refresh, single flight	EXEC

Project — Wire inspector. Encode one message by hand, verify it byte for byte against the library's output, then capture the HTTP/2 frames that carry it.

Path capstone — Production API Layer. A reusable network package plus a client app with auth, pagination, caching, backoff, cancellation, tests and logging. Where the app speaks both REST and gRPC, they share one token vault.

Path 7 — Persistence and Databases (26 lessons)

Every lesson here answers: What survives process death? What consistency guarantee is needed? What does this index or cache optimize, and what does it cost? How is stale data invalidated?

7A — Files and serialization

01	Volatile versus persistent	EXEC
02	Files and directories	EXEC
03	Serialization formats	EXEC
04	Atomic writes	EXEC
05	Corruption, and recovering from it	EXEC

Project — File-backed store. Serialize models with atomic replacement, and handle a corrupted file without losing the good data.

7B — Relational database fundamentals

01	Why databases exist	DOC
02	Tables, rows, relations	EXEC
03	Primary and foreign keys	EXEC
04	SQL basics	EXEC
05	Indexes	EXEC
06	Query plans	EXEC
07	Transactions	EXEC
08	ACID and normalization	EXEC

Project — SQLite notebook. A schema, CRUD, an indexed search you can time against an unindexed one, and a transaction rolled back on purpose.

7C — Apple persistence

01	SQLite under Core Data	EXEC
02	The Core Data stack	EXEC
03	Managed objects and faulting	EXEC
04	Contexts and threads	EXEC
05	Fetching, and NSFetchedResultsController	EXEC
06	Migrations	EXEC

Project — Persistent catalog app. An Apple persistence stack with migration-aware modeling, and a migration you actually run.

7D — Caching and offline design

01	Cache hit and miss	EXEC
02	Memory versus disk cache	EXEC
03	NSCache, and the decode tax	EXEC
04	TTL	EXEC
05	Invalidation	EXEC
06	Stale-while-revalidate	EXEC
07	LRU, and offline-first	EXEC

Project — Offline-first repository. Memory plus disk plus API, with stale data, revalidation, mutation invalidation and a stated failure policy.

Key question. Before choosing storage, ask: how much data, how structured, how sensitive, how often read and written, how long must it survive, and how consistent must it be?

Path capstone — Offline-First Data App. Database, two-level cache, repository abstraction, migrations, offline reads, invalidation and tests.

Path 8 — Software Engineering (41 lessons)

Every lesson here answers: What responsibility belongs here? Which way do the dependencies point? How can this be tested or replaced? Which tradeoff makes the design simpler or safer?

8A — Clean design and SOLID

01	Separation of concerns	DOC
02	Cohesion and coupling	EXEC
03	Single responsibility	EXEC
04	Open/closed	EXEC
05	Liskov substitution	EXEC
06	Interface segregation	EXEC
07	Dependency inversion	EXEC

Project — Refactor challenge. Turn a massive view controller into focused, testable components without changing behaviour.

8B — Architecture and state

01	MVC, and the massive view controller	EXEC
02	MVVM	EXEC
03	Coordinator	EXEC
04	Repository	EXEC
05	Explicit screen state	EXEC
06	State machines	EXEC
07	Derived state, and boolean explosion	EXEC

Project — Feature architecture project. One feature with explicit state, owned navigation and a repository boundary.

8C — Dependency injection and patterns

01	Constructor injection	EXEC
02	Protocols as seams	EXEC
03	Factories	EXEC
04	Containers	EXEC
05	Delegate, observer, strategy, adapter	EXEC
06	The singleton tradeoff	EXEC

Project — Dependency composition root. Wire production and fake implementations without a service locator.

8D — Testing

01	What a unit test actually is	EXEC
02	Fakes, stubs and mocks	EXEC
03	XCTest	EXEC
04	Testing async code	EXEC
05	UI tests	DEV
06	What not to test	DOC

Project — Test harness. Tests for a view model, a repository, and every state transition and error path.

8E — Modularization and teamwork

01	Targets and modules	EXEC
02	Swift Package Manager	EXEC
03	Access control boundaries	EXEC
04	Dependency direction	EXEC
05	Git fundamentals	EXEC
06	Merge versus rebase, and code review	EXEC
07	CI/CD	EXEC
08	Feature flags and environment config	EXEC

Project — Modularize an app. Extract core, network and feature packages, and write down the dependency rules they must obey.

8F — Debugging, performance and observability

01	The debugger	EXEC
02	The Memory Graph	DEV
03	Time Profiler	DEV
04	Allocations and Leaks	EXEC
05	Structured logging	EXEC
06	Crash reports and symbolication	EXEC
07	Metrics, and the analytics boundary	DOC

Project — Performance clinic. Diagnose and fix an intentionally slow, leaky screen, with before-and-after evidence.

Path capstone — Production Refactor. Take a medium app from tightly coupled code to a modular, testable architecture with tests and profiling notes.

Path 9 — Apple Platform Internals (27 lessons)

Useful once the earlier layers are comfortable. The goal is conceptual fluency, not kernel-development expertise.

Every lesson here answers: Which Apple subsystem is involved? What observable behaviour proves the model? Where is the line between documented API and implementation detail? Which tool validates the hypothesis?

9A — Darwin/XNU and the process model

01	Darwin and XNU	DOC
02	Mach and BSD	DOC
03	How an installed app becomes a process	EXEC
04	The sandbox and containers	EXEC
05	Entitlements	EXEC
06	Code signing	EXEC

Project — Platform map. Document how a signed app on disk becomes a sandboxed running process.

9B — Loading and runtimes

01	The app bundle	EXEC
02	The binary and its frameworks	EXEC
03	dyld and the shared cache	EXEC
04	The Objective-C runtime in production	EXEC
05	The Swift runtime and its metadata	EXEC

Project — Launch trace. Binary to dyld to frameworks to runtime to the first scene, documented.

9C — Run loop, GCD and lifecycle internals

01	CFRunLoop	EXEC
02	Dispatch internals	DOC
03	The launch sequence	DEV
04	Suspension and background execution	EXEC
05	Memory pressure and jetsam in production	DOC

Project — Runtime behavior monitor. Timers, events, background transitions, lifecycle and memory observations in one app.

9D — Rendering and diagnostics

01	The Core Animation render server	DOC
02	Compositing	DEV
03	Instruments	DEV
04	os_signpost	EXEC
05	Reading a crash report	EXEC

Project — Frame-to-pixel report. Profile an animated screen and explain the CPU, GPU and framework boundaries you measured.

NEW IN THIS EDITION

9E covers the device's sensors, left out of the first edition. Permission is a state machine, not a boolean, and getting that wrong is one of the most common sources of shipped bugs in apps that use location or notifications.

9E — The device's senses

01	CoreLocation: permission as a state machine	EXEC
02	Accuracy, privacy, and the reduced-accuracy path	EXEC
03	Maps and geocoding: degrees, cameras, names	EXEC
04	APNs from the socket up	EXEC
05	Notification routing and service extensions	DEV
06	LocalAuthentication and the Keychain gate	EXEC

Project — Sensor diagnostic screen. One screen that reports every permission state honestly, including the states you cannot reach without a device, and explains what the app should do in each.

Avoid rabbit holes. Learn enough internals to explain observable app behaviour. Do not spend weeks on Mach messages or compiler SIL before the OS, Swift, UIKit, concurrency, networking and database fundamentals are solid.

Path capstone — iOS Internals Field Guide. Instrument a real app and produce a technical report tracing launch, events, concurrency, memory, rendering, signing and lifecycle.

Connection maps

How familiar iOS concepts trace downward through the stack.

iOS concept	Trace it downward
weak self	closure capture → class reference → ARC → retain cycle → weak reference → object lifetime
UITableView cell reuse	object allocation → memory pressure → reuse pool → scrolling performance → main-thread layout and rendering
DispatchQueue.main.async	OS thread → main thread → event loop / run loop → queued closure → UIKit-safe state mutation
@MainActor	language and runtime isolation rule → executor scheduling → serialized access to UI-bound state
Codable decoding	network bytes → Data → parser/decoder → type-checking assumptions → allocations → model values
Core Data context	persistence model → context / unit of work → object graph → thread confinement → save transaction
Diffable data source	Hashable identity → snapshot / state → diff algorithm → batched UI update → animation
Auto Layout	constraint equations → layout pass → frame calculation → layer geometry → rendering
A stack view	design decomposition → arranged subviews → implicit constraints → the same layout pass → geometry
A gRPC call	generated stub → protobuf encoding → HTTP/2 stream → frames → TLS record → TCP segment
EXC_BAD_ACCESS	invalid memory access → process protection → kernel exception → debugger / crash report
App backgrounding	OS lifecycle decision → scene / app delegate callbacks → suspension limits → task cancellation and persistence decisions

One complete trace: API response to pixels

```
User taps refresh
→ UIKit event handling
→ Task begins
→ URLSession request
→ DNS / TCP / TLS / HTTP
→ server response bytes
→ Data
→ JSONDecoder
→ Swift model values
→ repository / cache decision
→ ViewModel state
→ @MainActor / UI update
→ diffable snapshot / layout
→ CALayer / Core Animation
→ GPU compositing
→ pixels
```

Study method

Because you already build iOS features, the fastest route is not to redo beginner tutorials. Use your existing experience as retrieval cues.

Step	What to do	Example
1. Learn terminology	Read and define the terms precisely	process vs thread vs task
2. Build a mental model	Draw the components and arrows	Task → executor → thread → CPU
3. Connect to iOS	Find where you have already used it	Task { } in a view controller
4. Run an experiment	Change one variable and observe	block the main thread, watch the UI freeze
5. Explain it	Write 5–10 sentences without notes	why UI must not be blocked
6. Debug it	Study a realistic failure mode	race, retain cycle, stale cache
7. Revisit	Review on a spaced schedule	1, 3, 7, 21 days

Weekly rhythm at 15–20 hours

- **6–8 hours** — new lessons, at roughly one hour each
- **3–4 hours** — the chapter project in progress
- **2–3 hours** — experiments and debugging on your own code
- **1–2 hours** — scheduled review of earlier lessons
- **1–2 hours** — writing: explain-backs, connection notes, the design note for the current project

Output over consumption. Every week should produce something: a diagram, a small program, a debugging experiment, a one-page explanation, or a refactor. Passive reading hours alone will not build the connections you want.

Spaced review

A lesson passed today comes back tomorrow, then after 3 days, then 7, then 21. A review that goes well moves the lesson up the ladder. A review that goes badly resets it to tomorrow. This is why one concept gets one lesson: a concept folded into another lesson never gets its own place in the schedule, and so never gets reviewed on its own.

Final integrated project

After the ten capstones, build one serious UIKit application without following a tutorial. The app matters less than the requirement that every subsystem forces you to use and explain the layers beneath it.

Required product capabilities

- Multi-screen navigation with adaptive Auto Layout, reusable list or grid UI, Dynamic Type and VoiceOver.
- A screen structure built from your own spacing and type scale, with all five screen states handled.
- An authenticated HTTP API with typed requests and responses, pagination, cancellation, a retry and backoff policy, and meaningful error states.
- Local persistence plus a two-level cache with explicit invalidation and stale-while-revalidate behaviour.
- Offline-first first paint: cached data stays usable while revalidation runs.
- Swift Concurrency: Task, async/await, structured concurrency, @MainActor for UI state, actors and Sendable for shared state.
- Explicit state with MVVM or a similar presentation pattern, Repository, dependency injection, and Coordinator where useful.
- Unit tests for business, state and repository behaviour, plus a few meaningful integration tests.
- Structured logging with enough diagnostics to understand network, persistence, state transitions and failures.
- At least one performance profiling pass and one memory-leak verification pass.

Required scenarios

Scenario	What you must demonstrate
Cold launch	Explain process and app launch, loading, the initial storage read, the network decision, and first render.
Refresh	Trace tap → responder / run loop → Task → URLSession → decode → repository / cache → MainActor → diff, layout, render.
Slow network	UI stays responsive, cached state remains, the request can be cancelled, loading semantics are explicit.
Concurrent requests	Duplicate work is prevented or deliberately coordinated; shared mutable state is race-safe.
Offline mode	Useful cached data remains; the error state explains the failure to revalidate without destroying state.
Mutation	Create, edit and delete cause correct cache, database and network invalidation and UI reconciliation.
Memory lifecycle	Controllers, tasks, closures and cache objects deallocate when expected; no unexplained retain cycle.
Large data set	Pagination, list reuse, image work, database indexing and rendering all stay performant.
Accessibility	Core workflows work with VoiceOver and at the largest Dynamic Type setting.
Failure injection	Decoding, server, auth, storage and cancellation failures, each explained by the layer that owns it.

Deliverables

- Running app and source code.
- Architecture diagram and module dependency map.
- A “request to pixels” trace for one core feature.
- A “tap to machine” trace covering UIKit, concurrency, threads, memory, network, database and rendering boundaries.
- The test suite, plus a short statement of what is deliberately not tested.
- A performance and memory report with real measurements.
- A failure-mode table: symptom → responsible layer → diagnosis → fix.
- A retrospective: what you used to do by habit, what you can now explain from first principles, and what gaps remain.

Graduation criterion. You do not need to remember every implementation detail. You should be able to locate a problem in the correct layer, explain the major mechanisms accurately, design an experiment to validate your hypothesis, and read deeper documentation without the terminology feeling disconnected.

Sequence and budget

Recommended order

Phase	Focus
Phase A — Machine & OS	Path 0 → Path 1
Phase B — Language machinery	Path 2 → Path 3
Phase C — iOS execution model	Path 4
Phase D — Concurrent & distributed work	Path 5 → Path 6
Phase E — Data	Path 7
Phase F — Engineering at scale	Path 8
Phase G — Apple internals	Path 9

Paths 0, 1, 2 and 3 are load-bearing. Nothing above them works without them, and they are the ones most likely to be skipped because they feel furthest from the work. Do not skip them.

What the whole plan costs

Work	Count	Each	Hours
Lessons	352	~1 h	352
Chapter projects	48	3–4 h	~170
Path capstones	10	8–12 h	~100
Final integrated project	1	50–70 h	~60
Scheduled review	—	~15% of lesson time	~55
Total			~740 h

At 15–20 hours a week, year one is 780 to 1,040 hours. The whole plan fits inside year one at the top of that range, and runs a few months into year two at the bottom. Nothing needs to be cut.

A rough two-year shape

Period	Paths	Lessons	Hours
Year 1, first half	0, 1, 2	82	~180
Year 1, second half	3, 4	100	~230
Year 2, first half	5, 6, 7	102	~230
Year 2, second half	8, 9, final project	68	~200

Year one ends with the machine, the OS, the compiler, Swift and UIKit installed — already past the point where the terminology stops feeling disconnected.

If it runs long, cut whole chapters, not lesson depth. Merging lessons makes each one shallower and costs it its own place in the review schedule. Paths 8 and 9 are the safest to defer. The projects are not: they are a third of the hours and they are the part that actually installs the skill. Under time pressure the temptation is to keep reading and skip building, because reading feels like progress. Lessons without their projects produce exactly the recognition-level knowledge this plan exists to fix.

Master checklist

Area	Coverage	Status
Computer systems	binary, CPU, instructions, registers, stack and heap, memory hierarchy, program execution	☐ Not started ☐ Learning ☐ Can explain
Operating systems	processes, threads, scheduling, virtual memory, syscalls, files, sandbox, lifecycle	☐ Not started ☐ Learning ☐ Can explain
Compiler / runtime	compiler phases, linking, loading, ARC, runtime metadata, dispatch	☐ Not started ☐ Learning ☐ Can explain
Swift	basics through protocols, generics, ownership, ARC, type system	☐ Not started ☐ Learning ☐ Can explain
UIKit	views and controllers, Auto Layout, navigation, lists, responder chain, run loop, rendering	☐ Not started ☐ Learning ☐ Can explain
Design and composition	units, scales, colour, components, states, screen decomposition, design systems	☐ Not started ☐ Learning ☐ Can explain
Interface Builder / localization	storyboards, XIBs, traits, strings tables, plurals, runtime switching	☐ Not started ☐ Learning ☐ Can explain
Concurrency	threads and tasks, GCD, async/await, structured concurrency, actors, Sendable, cancellation	☐ Not started ☐ Learning ☐ Can explain
Networking	DNS, TCP, TLS, HTTP, URLSession, retries, caching, auth	☐ Not started ☐ Learning ☐ Can explain
Wire protocols	protobuf, HTTP/2, gRPC calls, streaming, interceptors, pinning	☐ Not started ☐ Learning ☐ Can explain
Persistence	files, SQL, indexes, transactions, Core Data, cache design	☐ Not started ☐ Learning ☐ Can explain
Engineering	SOLID, MVVM, DI, repository, state, testing, modularization, Git, CI/CD	☐ Not started ☐ Learning ☐ Can explain
Performance / debugging	debugger, Memory Graph, Instruments, logging, crash reports	☐ Not started ☐ Learning ☐ Can explain
Apple internals	Darwin/XNU, dyld, runtime, code signing, entitlements, memory pressure	☐ Not started ☐ Learning ☐ Can explain
Device sensors	location, maps, APNs, notifications, biometrics	☐ Not started ☐ Learning ☐ Can explain

Final principle

The goal is not to know every API. The goal is to build a connected model of how software works, so that unfamiliar APIs become easier to understand, debug and evaluate.